

PROJECT WORK

Bayesian Optimisation with Differentiable Simulated Priors for Particle Accelerator Tuning

Author:

Hiba Sikander

Supervisor: Jan Kaiser, M.Sc.

Jannis Lübsen, M.Sc.

Examiner: Prof. Dr.-Ing. Annika Eichler

March 2, 2026

Project Work

for Ms. Hiba Sikander

PA 25 - 08

Student ID: 610416

Study Course: M.Sc.
Data Science

Title:**Bayesian Optimisation with Differentiable Simulated Priors for Particle Accelerator Tuning****Project Description:**

The tuning of particle accelerators is a time-consuming and difficult task that continues to be performed manually in many cases, reducing the scientific output of some of the most advanced but also costly research facilities in the world. In the field of autonomous accelerator tuning, researchers aim to develop methods that assist human operators and thereby reduce tuning times and extend the experimental capabilities of accelerators. One of the most promising approaches to solve the autonomous accelerator tuning problem is the use of Bayesian Optimisation (BO), which is already successfully used in operations. [1]

However, the most interesting accelerator tuning tasks are characterised by their high dimensionality. Despite being more capable at dealing with many dimensions than other algorithms, BO struggles to solve optimisations problems with more than a few dozen dimensions. To solve this challenge, researchers have proposed to leverage domain knowledge by replacing the commonly used but uninformative zero mean prior with a model of the system under optimisation. So far neural networks have been used for this purpose as they are differentiable and fast to compute. [2] While this work has shown the advantages of using an informed neural network prior, neural networks come with the disadvantage that they require large amounts of data to be trained, and that outside of the training data domain, their behaviour is ill-defined. In recent developments, high-speed differentiable simulators such as Cheetah [3] have been proposed as an alternative for use as a prior in BO, with the key advantage that they do not require any training data, and are well-behaved throughout. [3]

The goal of this research project is to take the BO with Cheetah prior approach presented on a very simple example in [3], and implement it on the closer-to-real world as well as more complex transverse beam parameter tuning task at the ARES accelerator, which has already been extensively studied with other autonomous tuning methods. The developed implementation should then be evaluated according to previous evaluations, to establish how this improved variation of BO compares to other methods and whether extension to even more complex accelerator tuning tasks is a promising avenue for future research.

Tasks:

1. Implement BO with Cheetah prior for tuning the transverse beam parameters on a simulation of the ARES Experimental Area.
2. Evaluate the BO with Cheetah prior implementation in accordance with the evaluation in [4].
3. Evaluate timing and speed of BO with an uninformed prior vs. BO with a Cheetah informed prior.
4. Study performance under model uncertainties.
5. **(Optional depending on accelerator availability)** Evaluate the developed implementation on the real ARES accelerator in accordance with [4].
6. **(Optional if time allows)** Setup a general implementation of BO with Cheetah.

References:

- [1] R. Roussel et al., “Bayesian optimization algorithms for accelerator physics”, *Physical review accelerators and beams*, 2024.
- [2] T. Boltz et al., “Leveraging prior mean modules for faster bayesian optimization of particle accelerators”, *Scientific Reports*, 2025.
- [3] J. Kaiser et al., “Bridging the gap between machine learning and particle accelerator physics with high-speed, differential simulations”, *Physical review accelerators and beams*, 2024.
- [4] J. Kaiser et al., “Reinforcement learning-trained optimisers and bayesian optimization for online particle accelerator tuning”, *Scientific reports*, 2024.

Supervisor: Jan Kaiser, M.Sc.
Jannis Lübsen, M.Sc.

Examiner: 1. Prof. Dr.-Ing. Annika Eichler

External: Deutsches Elektronen-Synchrotron (DESY)

Deutscher Titel:

Start Date: 17.11.2025

Due Date: 02.03.2026

March 2, 2026, Prof. Dr.-Ing. Annika Eichler

Hereby I declare that I produced the present work myself only with the help of the indicated aids and sources.

Hamburg, March 2, 2026

Hiba Sikander

Abstract

Tuning particle accelerators is an expensive and time consuming challenge that causes limitations in the findings of advanced research facilities. Bayesian Optimisation (BO) has shown potential to be a promising approach for autonomous tuning of accelerators, yet its effectiveness diminishes for higher dimensional problems. Recent studies have proposed replacing the zero mean prior in BO with physics-informed prior models to leverage physics domain knowledge. Cheetah framework, a high-speed differentiable simulator, was demonstrated as a physics-informed BO prior on a simple two dimensional FODO lattice. This project work extends that approach to a real world five dimensional problem, which is the transverse beam parameter optimisation at ARES Experimental Area involving three quadrupole magnets and two corrector magnets. This project implements a framework that incorporates Cheetah as the Gaussian Process mean function into the Xopt optimisation toolset with trainable misalignment parameters that enable the prior to adjust when the simulator does not accurately represent the real system. Four optimisation approaches (BO matched prior, BO mismatched prior, standard BO and Nelder-Mead) are evaluated. The results show that both physics-informed BO variants consistently achieve better performance in fewer iterations than both baseline methods, with the matched prior yielding the best overall performance across all evaluated metrics and BO with physics-informed prior remains effective even under prior-model mismatch. By significantly reducing the time and effort required for beam tuning, this work enables particle accelerator facilities to dedicate more time to experiments, pursue a wider range of research and ultimately enhance their contribution to more scientific progress.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Extension From Simple FODO To Complex ARES Problem	2
1.3	Contribution	4
2	Theory	5
2.1	Gaussian Process	5
2.2	Bayesian Optimisation	5
2.3	From Uninformed to Physics-Informed Prior	7
2.4	Cheetah-Based Differential Simulation Library	8
2.5	The ARES Accelerator	9
3	Implementation	11
3.1	Initial Setup	11
3.2	Physics-Informed Prior Mean Module	13
3.3	Optimisation Runner	15
3.4	Evaluation Metrics	18
4	Result and Discussion	19
4.1	Final Results	19
4.2	Discussion	23
5	Conclusion and Outlook	25
6	Future Work	26

1 Introduction

1.1 Motivation

Particle accelerators are one of the most advanced scientific devices ever developed to perform innovative scientific experiments. The Deutsches Elektronen-Synchrotron (DESY) [1] located in Hamburg hosts multiple leading accelerator facilities such as European X-ray Free Electron Laser (XFEL). These facilities accelerate charged particles, that enable researchers to study atomic level processes and structures with great precision. A crucial challenge for particle accelerators is the tuning process, which involves adjustment of many magnetic components to produce desired beam properties including position, size and divergence. As a specialised test facility for accelerators research and development, Accelerator Research Experiment at SINBAD (ARES), which is a linear accelerator at DESY, allows the evaluation of innovative tuning techniques prior to deployment on even larger facilities.

Beam tuning is essentially an optimisation task. In order to find exact configurations that minimise the deviation between the actual beam properties and the desired beam, we optimise the adjustable parameters (like magnet strengths and corrector/steering angles). However, the optimisation is not that simple and perhaps contains a number of challenges:

1. **Expensive Evaluation:** It is impractical to conduct an extensive search since measurement of each beam property takes time and resources.
2. **Uncertainty:** There are a number of systematic errors and noises that might affect beam measurement.
3. **Black-box Nature:** The complex relationship between input parameters and beam characteristics cannot be directly observed analytically during operation.
4. **Unknown / Partial Observability:** Important parameters like characteristics of incoming beam and magnet misalignments are frequently either unknown or partially observed.

A traditional approach to accelerator tuning could be through skilled professionals who manually adjust parameters using their knowledge and experience. This method significantly relies on individual skill, takes a lot of time, is expensive and does not scale well to more complicated accelerator systems. Therefore, automated optimisation methods are necessary for effective accelerator performance. Several optimisation algorithms have been applied to particle accelerator tuning problems with their own unique advantages and disadvantages. Nelder-Mead Simplex (NM) is a gradient-free optimisation technique and one of the approaches that is widely used because of its simplicity and robustness [2]. However, NM is best suited for convex optimisation landscapes and tends to struggle in higher-dimensional search spaces. Another approach is the Reinforcement Learning (RL) technique that has shown remarkable performance [3]. RL agents can learn to tune accelerators in very few steps during deployment by training neural network policies in

simulation. However, RL requires extensive training data as well as very careful reward function design. The trained policies could not generalise well to conditions outside of their training distribution. The approach of our interest for this project work is Bayesian Optimisation (BO), which finds a middle ground among these approaches. BO creates a probabilistic surrogate model of the objective function, which is usually a Gaussian Process (GP), that provides predictions as well as estimates of uncertainty. This makes it possible for the assessment points to wisely balance between both exploration and exploitation. BO is a desirable option for accelerator tuning since it excels at solving expensive black-box optimisation problems and is already used well with great success on accelerators [4–6] .

Another key insight motivating this project work is that, since the underlying physics of particle accelerators is well known, this means we can build reliable models based on physics knowledge. Thus, all thanks to modern simulation library like Cheetah [7] that takes advantage of this, making it possible to predict beam properties quickly and accurate enough to be used directly within an optimisation loop. The standard BO uses uninformed zero mean prior function in the GP model. This implies that an algorithm must learn the whole function landscape from scratch and have to begin with no prior knowledge of the intended objective function values. On the other hand, if the physics-informed knowledge could be incorporated into the prior, this shall give advantage and optimisation could already start with meaningful predictions that could focus its limited evaluation budget on identifying the differences between the model and the reality. The importance of physics-informed prior for accelerator optimisation has also been shown in the recent work [8], where the neural network surrogate models trained on the simulation data can perform as effective prior mean functions, significantly speeding up the convergence. However, training neural network surrogates requires very large datasets from simulations.

Although, other simulators like Methodical Accelerator Design (MAD-X) [9] and ELEc-tron Generation ANd Tracking (elegant) [10] have been around for years and are capable of simulating beam dynamics, but they do not compute the gradients directly and must rely on numerical gradient estimation methods such as finite differences, this makes them unsuitable for integrating as GP mean function because it would be highly inefficient, since the surrogate model parameters must be updated at every optimisation step. Cheetah being built on PyTorch, can compute gradients directly through automatic differentiation, enabling it to be embedded as the GP mean function with its parameters optimised jointly alongside the surrogate model.

1.2 Extension From Simple FODO To Complex ARES Problem

Cheetah has already been demonstrated as a physics-informed prior for BO on a simple and straightforward beam focusing Focus-Drift-Defocus-Drift (FODO) lattice [7]. A FODO cell is a fundamental building block of an accelerator lattice design, which is made up of alternating quadrupole magnets for focusing and defocusing, that are separated by drift spaces.

Since this optimisation problem was two-dimensional, thus it only involved strengths of

two quadrupole magnets (k^{Q1} and k^{Q2}) with the objective of minimising beam size at target location, defined as

$$mae = 0.5(|\sigma_x| + |\sigma_y|), \quad (1.1)$$

where σ_x and σ_y are the horizontal and vertical beam sizes, considering only beam size (focusing) without beam position (centering), unlike ARES problem, reflecting the simpler nature of FODO demonstration. Moreover, although there were two drift spaces (d_1 and d_2), they were constrained to have same lengths between the quadrupoles, therefore only a single trainable parameter was there, which was restricted to the interval $[0.2, 1.0]$ m and was included in the physics-informed prior mean function. The results showed that BO with Cheetah prior converged to the true minimum within 15 steps, indicating better sample efficiency than NM and standard BO with zero prior. Furthermore, the method worked even with a mismatched prior, the algorithm learned the correct values during GP model fitting and converged to the true optimum when the drift lengths in the prior model (0.5m) was different from the ground truth (0.7m). While this validated the fundamental concept, the FODO problem is a simple example, which does not depict the complex nature of actual accelerator tuning operations. The two-dimensional search space is quite easy to explore and the model reality mismatch was restricted to a single geometric parameter (drift length) instead of the various other sources of uncertainty found in actual accelerators.

This project extends the FODO proof-of-concept to a significantly more complex and realistic problem which is transverse beam tuning at the ARES Experimental Area at DESY. This extension contains following key advancements:

1. **Higher Dimensionality:** In contrast to two-dimensional FODO problem, ARES problem is a five-dimensional search space, including two steering/corrector magnet angles (θ^{CV} and θ^{CH}) and three quadrupole strengths (k^{Q1}, k^{Q2}, k^{Q3}). The volume of the search space is greatly increased by 2.5x in dimensionality, which makes it more difficult and raises the importance of informed prior.
2. **Realistic Lattice:** ARES Experimental Area is a real accelerator section loaded from an actual lattice description file unlike the synthetic FODO cell.
3. **Objective Nature:** For ARES problem, the objective function combines four beam parameters into a single metric, including horizontal and vertical sizes (σ_x, σ_y) as well as the positions (μ_x, μ_y), simultaneously optimising beam focusing and centering.
4. **Trainable Quadrupole Misalignments:** In real accelerators, quadrupole magnets are never perfectly aligned and cannot be measured directly. That is why, it is interesting to use quadrupole misalignments as the cause of model reality mismatch under this project work. The ARES prior includes 6 trainable misalignment parameters (horizontal and vertical for each quadrupole Q1, Q2 and Q3), which are limited to physically plausible intervals of ± 0.5 mm, in contrast to the FODO demonstration, where only a single parameter (drift length) was learned.

1.3 Contribution

Thus, the main contributions of this work are :

1. Demonstrating that Cheetah-informed prior approach is scalable from 2D synthetic lattice to 5D real world accelerator tuning problem.
2. Implementing physics-informed prior mean function for ARES lattice, being compatible with Xopt Bayesian optimisation framework.
3. Extending prior with six trainable misalignment parameters, enabling the physics model to adapt to real accelerator conditions during optimisation.
4. Containing a comprehensive comparison between standard BO, NM and physics-informed BO (matched and mismatched prior) across multiple trials.

2 Theory

In this chapter, the theoretical foundations of this project work are developed. We begin with Gaussian process as a probabilistic function approximator, present Bayesian optimisation as a framework for sample efficient black-box optimisation and then look over the Cheetah simulator and ARES accelerator, showing how physics knowledge may be integrated through the prior mean function.

2.1 Gaussian Process

Gaussian process (GP) offers a robust framework for regression which allows uncertainty quantification along with predictions. GPs are well suited for modeling unknown objectives in optimisation problems [4], because they define probability distribution directly over functions, unlike parametric models that learn a predefined set of weights. A Gaussian process is fully specified by its mean function $m(x)$ and covariance function (kernel) $k(x, x')$, thus can be written as

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')), \quad (2.1)$$

which states that a function $f(x)$ is drawn from GP, where the mean function $m(x)$ encodes previous expectations regarding the behavior of the function and the kernel $k(x, x')$ carries assumptions regarding smoothness and correlation between points. The kernel selection determines what kind of functions the GP considers probable. Because of its ability in modeling smooth and physically realistic functions without assuming infinite differentiability, Matérn-5/2 kernel is widely used in Bayesian Optimisation [4]. The GP uses the Bayes's rule,

$$p(f|X, y) \propto p(y|f, X)p(f|X), \quad (2.2)$$

to update from prior to posterior when we observe data. $p(f|X, y)$ is the posterior (updated GP), $p(f|X)$ is prior (GP) and $p(y|f, X)$ is the likelihood of observed data. The posterior mean remains Gaussian, with mean and variance that reflects both our prior assumption and observed evidence. One of the important characteristics is that posterior uncertainty is low in areas with dense observations and high in areas with sparse data, this natural uncertainty quantification is what makes GPs useful for guiding optimisation.

2.2 Bayesian Optimisation

Bayesian Optimisation (BO) is a sequential method for determining the optimal value of expensive black-box functions [4]. BO still works well when the gradient information is not available and each function evaluation is expensive. Because of these characteristics, BO is ideally suited for accelerator tuning, where it has emerged as a state-of-the-art method [5, 6]. The goal is to solve,

$$x^* = \arg \min_{x \in \mathcal{X}} f(x), \quad (2.3)$$

x represents the control variables (like magnet settings), $\mathcal{X}$ is the search space and $f(x)$ is the quantity to be minimised (beam error in our case). BO aims to find x^* using as few function evaluation as possible. The key idea is to use a probabilistic surrogate model (usually a GP) to guide the search rather than evaluating the expensive function everywhere. As shown in Figure 2.1, the optimisation process is iterative. We start with a modest initial set of observations, fit a GP model to the data and then choose the next point to evaluate using an acquisition function. Exploration (sampling when uncertainty is large) and exploitation (sampling where the predicted value is good) are balanced by the acquisition function. After evaluating the true objective at the selected point, we update the dataset and repeat until the evaluation budget is exhausted.

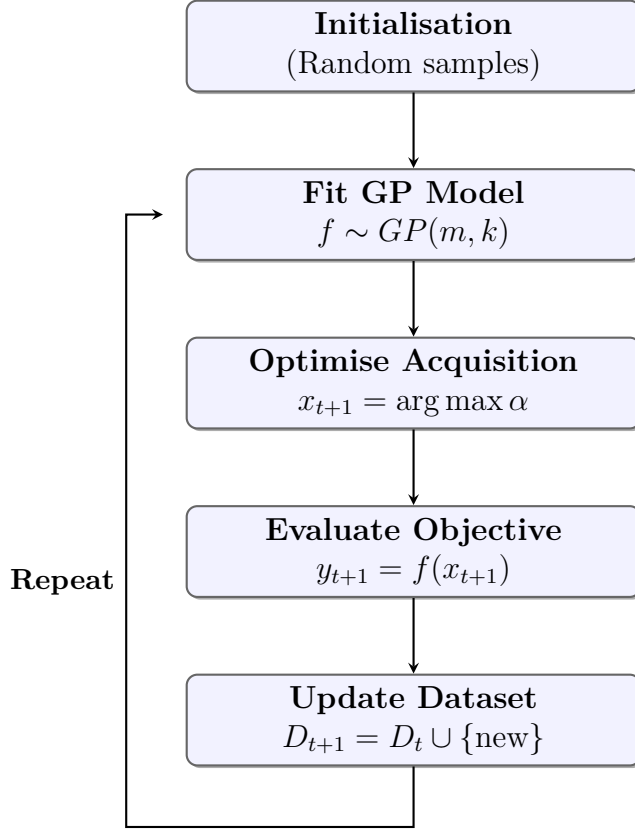


Figure 2.1: Bayesian optimisation loop.

Acquisition functions guide where to sample next by balancing two competing goals, where one is exploring uncertain regions while the other is exploiting areas that look promising. Two widely used acquisition functions are Expected Improvement (EI) and Upper Confidence Bound (UCB).

- **Expected Improvement:** This acquisition function computes how much better do we expect a candidate point to be compared to our current best observation

$f(x^*)$, the expectation is taken over the GP’s predictive distribution. EI can be mathematically defined as

$$\alpha_{EI}(x) = \mathbb{E}[\max(0, f(x^*) - f(x))], \quad (2.4)$$

where $f(x^*)$ is the best (lowest) observed value so far. The expectation averages over what the GP thinks might happen at x and the max ignores cases where things get worse by ensuring that only improvements are counted. EI naturally trades off exploration and exploitation without any tuning parameter.

- **Upper Confidence Bound:** This acquisition function takes more direct approach by simply combining the predicted model and uncertainty. UCB can be mathematically defined as

$$\alpha_{UCB}(x) = -\mu(x) + \beta\sigma(x), \quad (2.5)$$

where $\mu(x)$ and $\sigma(x)$ are the GP posterior mean and standard deviation respectively, and parameter β controls the exploration-exploitation tradeoff.

This project work, uses the Upper Confidence Bound (UCB) acquisition function (Equation 2.5) with $\beta = 2.0$, following the approach used in the first demonstration over simple FODO problem [7] and common practice in accelerator optimisation [4].

2.3 From Uninformed to Physics-Informed Prior

Before any data is observed, our belief about the objective is represented by the prior mean function $\mu(x)$. The standard practice is to use zero or constant mean, which assumes no prior knowledge. Physics models that approximate the system are often used in scientific applications and we have seen that incorporating this knowledge can significantly improve the performance [8]. To understand why, consider the GP posterior mean prediction at a new point. The GP learns to predict the difference between the prior mean and the actual function. These residuals are small and can be learned from fewer observations when the prior mean is accurate. Whereas, when the prior is simply zero, the GP has to learn the entire function from scratch. This motivates using a physics-informed prior mean function, defined as

$$m(x) = f_{physics}(x; \phi), \quad (2.6)$$

with $f_{physics}$ being the physics model prediction, x being input to the optimisation, which in our case are the five magnet settings required to be optimised (k^{Q1} , k^{Q2} , θ^{CV} , k^{Q3} , θ^{CH}) and ϕ represents parameters of physics model that are uncertain or unknown like, quadrupole misalignments, beam characteristics, drift lengths etc. In this project, it has been chosen to focus on quadrupole misalignments because they are a major source of model reality mismatch in ARES accelerator tuning. This benefits to improve sample efficiency and having meaningful predictions before observing data. In standard GP practice, which is

$$\omega^* = \arg \max_{\omega} \log p(y|X, \omega), \quad (2.7)$$

every time the GP model is fitted, it optimises the kernel hyperparameters ω (length scales, signal variance, noise variance) by maximising the log marginal likelihood of observed data [williams2006gaussian]. A key innovation in this project work is making the physics model parameters ϕ (in our case quadrupole misalignments: Δ_{xQ1} , Δ_{yQ1} , Δ_{xQ2} , Δ_{yQ2} , Δ_{xQ3} , Δ_{yQ3}) trainable alongside the standard GP hyperparameters, and can be represented as

$$(\phi^*, \omega^*) = \arg \max_{\phi, \omega} \log p(y|X, \phi, \omega), \quad (2.8)$$

with y being the observed objective values, X being input points evaluated so far and ϕ being the physics model parameters. As more data is collected the learned ϕ gets closer to the true values so the physics model becomes more accurate eventually leading to better prediction and optimisation performance. By optimising $\log p(y|X, \phi, \omega)$, it measures how well the model explains the observed data. Note that ω is always optimised during standard GP fitting, while ϕ is only optimised when explicitly set as trainable. This approach allows the physics model to adapt better and gradually converge towards their true values, making the prior more accurate and speeding up the convergence by mitigating the mismatch between the model and reality.

2.4 Cheetah-Based Differential Simulation Library

Using physics simulator as a prior mean function requires both speed (so the simulator must be fast enough to call within each GP evaluation) and differentiability (in order to facilitate gradient-based hyperparameter optimisation). Cheetah [7], was developed to satisfy these needs. Cheetah makes a conscious trade-off, in order to achieve speed, it gives up some fidelity. Its main objective is to give machine learning applications a differentiable beam dynamics code with improved speed over existing simulation codes. This ability of differentiation in Cheetah is important for our approach because when GP fits its hyperparameter by maximising marginal likelihood, gradients flow through the Cheetah simulation to update the trainable misalignment parameter. Gradients of beam dynamics are often expensive to compute analytically or numerically [7] but Cheetah overcomes this by utilizing PyTorch automatic differentiation, which computes them numerically (to machine precision), while also being much faster. Additionally, Cheetah enables BO packages like BoTorch to optimise acquisition functions effectively. There are two beam representations provided by Cheetah.

- **Parameter Beam:** assumes a Gaussian beam and represents it using a seven-dimensional mean vector and covariance matrix, only these statistical moments are propagated through the lattice, enabling really fast simulation.

- **Particle Beam:** represents the beam as individual macro-particles, each described by seven-dimensional vector. This allows modeling of non-Gaussian distribution and bunch substructures, but at greater computational cost.

2.5 The ARES Accelerator

The Accelerator Research Experiment at SINBAD (ARES) facility at DESY serves as a testbed for advanced accelerator concepts and to test machine learning techniques on them. It is an S-band radio frequency linac with two independently powered travelling wave accelerating structures and a photoinjector. These structures can operate at energies up to 154 MeV. ARES's main goal is to create and investigate sub-femtosecond electron bunches at accelerated energy. For uses like radiation production by free electron lasers, the capacity to produce such brief bunches is highly desirable. ARES is also used for medical applications and for study and development of accelerator components [3].

This project work focuses on ARES experimental area. The beamline consists of three quadrupole magnets (Q1, Q2, Q3) and two corrector/steering magnets (CV for vertical steering and CH for horizontal steering), and a diagnostic screen where beam properties are measured. The magnets sequence of ARES Experimental Area, which is a subsection of ARES is depicted in figure 2.2, where two quadrupole magnets (Q1 and Q2) are followed by the vertical corrector magnet (CV), then the third quadrupole magnet (Q3) and finally the horizontal corrector magnet (CH) before reaching the screen. Each magnet is separated by a drift space ($d_1, d_2, d_3, d_4, d_5, d_6$) each having lengths within the range of 0.175 m to 0.450 m.

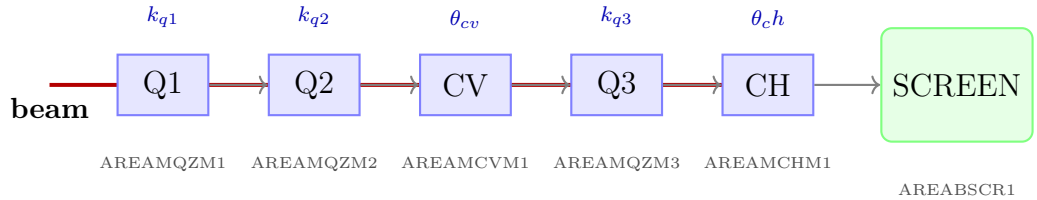


Figure 2.2: Schematic of the ARES Experimental Area showing the magnet sequence.

All magnets have polarity-switchable power supplies. It is possible to actuate the quadrupole magnets up to a 72 m^{-2} field strength. The steering/corrector magnets have a limit of 6.2 mrad. The camera's field of view is approximately 8 mm by 5 mm, with a pixel resolution of 2448 by 2040. The scintillating screen's effective resolution is about $20 \mu\text{m}$ [3]. The beam tuning task involves optimising the five magnets defined as,

$$\mathbf{u} = (k_{q1}, k_{q2}, \theta_{cv}, k_{q3}, \theta_{ch}), \quad (2.9)$$

where $\mathbf{u} \in \mathbb{R}^5$. In order to produce a focused and centred beam at screen, the measured observables are beam's horizontal and vertical positions (μ_x and μ_y) and sizes (σ_x and σ_y). The objective function used throughout this work is

$$mae = \frac{1}{4}(|\mu_x| + \sigma_x + |\mu_y| + \sigma_y), \quad (2.10)$$

which is the mean absolute error which combines both centering and focusing of the beam into a single metric to minimise. Where σ_x and σ_y are strictly positive, since they are beam sizes. For the target beam parameters, we consider all targets to be zero $(0, 0, 0, 0)$. In a real world accelerator, the quadrupole magnets are never exactly aligned, the precise beam distribution entering the experimental area is not known. Even with the corrector magnets set to zero, a misaligned quadrupole with transverse offset (Δ_x, Δ_y) partially functions as steering element, deflecting the beam. These misalignments are not directly measurable during operation. By treating the six misalignment parameters $(\Delta_{xQ1}, \Delta_{yQ1}, \Delta_{xQ2}, \Delta_{yQ2}, \Delta_{xQ3}, \Delta_{yQ3})$ as trainable quantities within the physics-informed prior, our approach addresses this issue too for the prior mismatched scenario and allows the model to adjust to actual accelerator settings using the optimisation data.

3 Implementation

This chapter covers the implementation of BO with a Cheetah-based physics-informed prior for the transverse beam tuning at the ARES Experimental Area. Instead of being one large script, the implementation code is systematically divided into Python modules, each having their distinct functions:

- **bo_cheetah_prior_ares.py** includes the simulated ARES environment `ares_problem()` and the GPyTorch compatible prior mean class `AresPriorMean` with its trainable misalignment parameters. This is where most of the physics lives.
- **eval_ares.py** constructs the optimisers (Standard BO, BO with Cheetah prior and Nelder-Mead) along with the experimental scenarios (matched, mismatched and matched_prior_newtask) and loops through the optimisation trials.
- **eval_ares_metrics.py** it computes the performance metrics and generates a final csv with all the metrics to compare the techniques.
- **plot_ares_results.py** generates plots for the results.

The plotting and metrics modules are then connected by the fifth script **run_evaluation.py** to run altogether. Table 1 shows the summary of primary libraries used.

Libraries	Purpose	Version
Cheetah	PyTorch based differentiable beam tracking simulator.	0.8.0
GPyTorch	Gaussian process framework, base class for prior mean module.	1.14.2
Xopt	BO loop, UCB acquisition Nelder-Mead baseline.	2.6.7
BoTorch	BO primitives used internally by Xopt.	0.16.0

Table 1: Primary libraries/frameworks used.

3.1 Initial Setup

Before any optimiser, we need something to optimise against, `bo_cheetah_prior_ares.py` contains the method `ares_problem()` which fulfills this purpose. A beam is propagated across a Cheetah model of the ARES Experimental Area using a set of magnet parameters, and the beam quality is then returned. In practice, every call to this function mimics what would happen if the user adjusted the magnet settings on the real machine and then looked at the diagnostic screen.

In this project, Cheetah’s `ParameterBeam` is used to describe beam entering the ARES Experimental Area. It is used simply because it is faster, which enables to run the

experiments more quickly and iterate on the implementation with shorter turnaround times. Also, note that Cheetah’s `ParticleBeam` could have also been just as well but with more computational cost. Since speed is not the most critical concern in this project, there is nothing holding us back from using either representation for this application. The ARES experimental area only involves linear optics, so both approaches get equivalent results. These values match the typical ARES operating conditions.

A JSON file describing the full ARES accelerator is used to load the lattice itself. The code extracts the sub-segment between AREASOLA1 (marker) and AREABSCR1 (screen) as only ARES Experimental Area is required. Listing 1 demonstrates the loading of the lattice and the assignment of the five magnet parameters prior to tracking.

Listing 1: ARES lattice loading and magnet parameters configuration.

```

1
2 # Load ARES lattice
3 ares_segment = cheetah.Segment.from_lattice_json("
    ARESlatticeStage3v1_9.json")
4 ares_ea = ares_segment.subcell("AREASOLA1", "AREABSCR1")
5
6 # Set magnet parameters
7 ares_ea.AREAMQZM1.k1 = torch.tensor(input_param["q1"])
8 ares_ea.AREAMQZM2.k1 = torch.tensor(input_param["q2"])
9 ares_ea.AREAMCVM1.angle = torch.tensor(input_param["cv"])
10 ares_ea.AREAMQZM3.k1 = torch.tensor(input_param["q3"])
11 ares_ea.AREAMCHM1.angle = torch.tensor(input_param["ch"])
12
13 if misalignment_config is not None:
14     for magnet_name, (dx, dy) in misalignment_config.items():
15         magnet = getattr(ares_ea, magnet_name)
16         magnet.misalignment = torch.tensor([dx, dy], dtype=
            torch.float32
17     )
18 else:
19     ares_ea.AREAMQZM1.misalignment = torch.tensor([0.0,
20     0.0])
21     ares_ea.AREAMQZM2.misalignment = torch.tensor([0.0,
22     0.0])
23     ares_ea.AREAMQZM3.misalignment = torch.tensor([0.0,
24     0.0])
25
26 out_beam = ares_ea(incoming_beam)

```

For the three quadrupole strengths (k^{Q1} , k^{Q2} , k^{Q3}) and corrector angles (θ^{CV} and θ^{CH}), we have reduced the range from physical magnet’s $\pm 72 \text{ m}^{-2}$ to our simulation’s $\pm 30 \text{ m}^{-2}$, and from $\pm 6.2 \text{ mrad}$ to $\pm 6.0 \text{ mrad}$ respectively because the optimisation consistently converges well within the narrower range and exploring beyond it offers no practical benefit to the tuning process, since during practical operation, these ranges are usually never exceeded. The code iterates through the dictionary and applies each (Δ_x, Δ_y) pair to the

corresponding quadrupoles when a misaligned configuration is provided. This enables us to model the realistic scenario in which magnets are not perfectly centred on the beam. To let the optimiser know how good or bad the result is, we need a single number. Following the previous FODO demonstration [7], for ARES Experimental Area, we use the mean absolute error as objective function defined in Equation 2.10, including each of the four beams observable, with μ_x and μ_y being the beam positions (ideally, zero means beam is centred) and σ_x and σ_y being the beam sizes (the smaller the better), simultaneously centering and focusing the beam. The function also returns the individual beam parameters in addition to MAE so that they can be logged for later analysis.

3.2 Physics-Informed Prior Mean Module

The heart of this project work is `AresPriorMean`, defined in `bo_cheetah_prior_ares.py`. Its role is to turn the Cheetah simulation into a format that GPyTorch can use as the mean function of GP. Instead of using GP prior (typically with zero mean assumption), it begins with the best estimates from the physics model and just needs to learn the residual, which is the difference between what the simulator predicts and what is actually observed.

`AresPriorMean` class specifically inherits from `gpytorch.means.Mean` and implements a `forward()` method. Everytime GPyTorch needs the prior mean at a set of input points, it calls this method. The remaining GP components, such as kernel evaluation, posterior conditioning and acquisition function optimisation, perform exactly like they would with any other mean function. Another important aspect is that Xopt builds the input tensor for the GP by sorting variables in alphabetical order, which means that since our magnet variables are named `q1`, `q2`, `cv`, `q3` and `ch`, column 0 of the tensor would always be `ch`, not `q1` as one might expect from the beamline order. Listing 2 shows how the class defines explicit index constants in order to avoid silently feeding the wrong values to incorrect magnets.

Listing 2: Explicit indexing to ensure correct mapping in Xopt input tensor.

```

1
2 # Load ARES lattice
3 class AresPriorMean(Mean):
4     # Considering alphabetical order
5     VAR_ORDER = ['ch', 'cv', 'q1', 'q2', 'q3']
6     IDX_CH = 0
7     IDX_CV = 1
8     IDX_Q1 = 2
9     IDX_Q2 = 3
10    IDX_Q3 = 4

```

As discussed earlier, in real accelerators, the quadrupole magnets are never perfectly aligned. A magnet that is even with a fraction of a millimeter, deflects the beam in a way that would not be predicted by a perfectly aligned simulation, partially acting as a steering factor. In order to handle this, the prior module treats the quadrupole

misalignments as learnable parameters (Δ_{xQ1} , Δ_{yQ1} , Δ_{xQ2} , Δ_{yQ2} , Δ_{xQ3} , Δ_{yQ3}). Each of the three quadrupoles have their respective horizontal and vertical misalignment values, making a total of six values. In order to do online system identification, GPyTorch’s marginal likelihood maximization adjusts these values in addition to the standard GP hyperparameters as the optimiser collects data. Three coordinated layers are used to register each misalignment parameter, these three layers are summarized in Table 2. On top of this, *@property* accessors offer a convenient interface, reading a misalignment property applies the forward transform (raw to constrained) and when it is written, the inverse transform is applied. This three layer pattern is repeated identically for each of the six parameters. The constructor is fairly lengthy because of this six times repeated pattern, although a loop based factory would have been cleaner, but we go with the explicit version since it has the advantage of being easy to read and to debug.

Component	GPyTorch API	What it does
Raw Parameter	<code>register_parameter()</code>	A raw unconstrained <code>nn.Parameter</code> is created at first, which is what PyTorch’s optimiser updates directly.
Interval Constraint	<code>register_constraint()</code>	Secondly, an interval constraint is used that maps this raw value through a sigmoid based transformation into the physically meaningful range of ± 0.5 mm, this serves as the bound guaranteeing that the learnt offset can never be greater than what is practical for quadrupole misalignments.
Smoothed box prior	<code>register_prior()</code>	Third, a <code>SmoothBoxPrior</code> is registered across the confined space, adding a soft regularization on top of the hard constraint. This helps the optimiser avoid getting stuck at the boundaries by gently penalizing values near the edges while assigning a fairly uniform probability throughout the interval.

Table 2: Three layered pattern to register misalignments as trainable parameters.

When `forward()` method is called, it receives a tensor `X` whose last dimension contains 5 columns (the five magnets, in alphabetical order), the steps include:

1. Using the index constants from Listing 2, slice out each magnet parameter.
2. Assign them to corresponding components of Cheetah lattice.
3. Apply the current misalignment values to the three quadrupoles by stacking them.
4. Run the beam through the lattice
5. Return the MAE (Equation 2.10).

Since all of these operations (the beam tracking, the absolute values, and the summation) are native PyTorch operations, the full forward pass is differentiable. Gradients could flow backward through the simulator and into the six misalignment parameters, which is what allows the joint optimisation of both misalignment values as well as usual GP hyperparameters.

3.3 Optimisation Runner

After setting up the environment and the prior setup, the remaining part is the runner that actually executes the experiments. This is handled by an `eval_ares.py` file that takes arguments for the experimental scenarios (matched, mismatched, matched_prior_newtask), the number of trials, the evaluation budget and the type of optimiser (BO, BO_prior, NM) where BO is the standard Bayesian Optimisation, BO_prior is Bayesian Optimisation with physics prior and NM is Nelder-Mead. Xopt defines the optimisation problem declaratively using a YAML based specification known as VOCS (Variable and Objective Configuration Specification). Listing 3 shows the five magnet’s ranges and a single objective.

Listing 3: VOCS configuration.

```

1 vocs_config = """
2     variables:
3         q1: [-30, 30]
4         q2: [-30, 30]
5         cv: [-0.006, 0.006]
6         q3: [-30, 30]
7         ch: [-0.006, 0.006]
8     objectives:
9         mae: minimize
10 """
11 vocs = VOCS.from_yaml(vocs_config)

```

To test the prior under various conditions, the runner supports three scenarios given below, they are also summarised in Table 4:

1. **Matched Scenario:** This is the most simple and ideal scenario where the beam and misalignments in the environment are exactly what the prior expects, which are the default values (including all misalignment values to be zero). Thus, physics model perfectly matches the reality. This scenario exists mainly to verify that the implementation works correctly and to see how much the prior helps when it is perfectly accurate since there is no model reality gap at all. In practice, this situation never happens on a real machine.
2. **Mismatched Scenario:** In this scenario, a gap is introduced on purpose to create mismatch between the prior and the reality, a different beam (σ_y changed from $200\mu\text{m}$ to $100\mu\text{m}$), and non zero misalignment values over all three quadrupoles

are used, shown in Table 3. The prior, however, starts from the default beam and zero misalignments as it does not know about any of these changes. We tell Xopt to include the six misalignment parameters in the GP’s marginal likelihood optimisation by setting `trainable_mean_keys=["mae"]` in the `StandardModelConstructor` so they can be learned from data as it is received. This is the scenario that tests whether online systems identification actually works.

Magnet	Δ_x (mm)	Δ_y (mm)
AREASMQZM1	0.000	+0.200
AREASMQZM2	+0.100	-0.300
AREASMQZM3	-0.100	+0.150

Table 3: Ground truth quadrupole misalignments.

3. **Matched_prior_newtask Scenario:** The simulated accelerator setup is the same as in the mismatched scenario (modified beam and non-zero misalignments), but this time the prior is initialised with the correct values which are the right beam parameters and the true misalignment values. So this way, we are handing the prior a perfectly adjusted physics model and asking how fast can the optimiser converge when it already knows the true system’s state, compared to having to learn it during the process.

For the evaluation in this project, the `matched_prior_newtask` serves as our actual matched case, since it uses the realistic accelerator setup having modified beam and misalignments rather than the ideal zero misalignment defaults. We compare four methods, Nelder-Mead, Standard BO (zero mean), BO with Cheetah prior (mismatched scenario) and BO with Cheetah prior (matched_prior_newtask scenario). All four approaches face the same simulated accelerator so that the comparison is fair, what differs between them is only the optimisation strategy, and in case of prior informed methods, how much the prior knowledge is available from the start for each method.

Listing 4 shows how the most complex one (BO with mismatched prior) is set up. the others (NM, standard BO and BO matched prior) are the simpler subsets of the same pattern.

Listing 4: BO with Cheetah prior(mismatched setup).

```

1 # Create the prior mean module with default
2 prior_mean_module = bo_cheetah_prior.AresPriorMean(
3     incoming_beam=incoming_beam
4 )
5 # Initialise misalignments to zero, will be
  learned
6 prior_mean_module.q1_misalign_x = 0.0
7 prior_mean_module.q1_misalign_y = 0.0
8 prior_mean_module.q2_misalign_x = 0.0
9 prior_mean_module.q2_misalign_y = 0.0

```



```

10     prior_mean_module.q3_misalign_x = 0.0
11     prior_mean_module.q3_misalign_y = 0.0
12
13     #Inject into GP constructor with
14     trainable_mean_keys
15     gp_constructor = StandardModelConstructor(
16         mean_modules={"mae": prior_mean_module},
17         trainable_mean_keys=["mae"],
18     )
19
20     # Create UCB generator with beta 2.0
21     generator = UpperConfidenceBoundGenerator(
22         beta=2.0, vocs=vocs, gp_constructor=
23         gp_constructor
24     )

```

Scenario	Modified Beam	True Misalign- ments	Prior Misalign- ments	Trainable ϕ
Matched	—	—	—	—
Mismatched	✓	✓	0	✓
Matched prior (new task)	✓	✓	= true	—

Table 4: Three experimental scenario summary.

In order to have a fair comparison, every trial starts from the same initial points, which are

$$x_0 = (k^{Q1} = 10.0, k^{Q2} = -10.0, \theta^{CV} = 0.0, k^{Q3} = 10.0, \theta^{CH} = 0.0). \quad (3.1)$$

After this initial evaluation, the runner loops through the `xopt.step()` for a fixed number of iterations. The default in the code is 50 steps, but for the evaluation presented in this report, we used 100 steps to give each method more room to converge and to better observe the later stage behaviors of the optimisers. Each step involves fitting the GP to all the data collected so far, selecting next point by optimising the UCB acquisition function and testing that point on the simulated accelerator. In the case of Nelder-Mead, the same `step()` interface updates the simplex instead. For mismatched scenario when the prior’s misalignment parameters are trainable, fitting the GP also involves optimising over ϕ . As a result, each new data point gives the marginal likelihood optimiser more information about where the magnets are actually positioned and the prior gradually gets more accurate which is the online system identification already discussed above. After each trial completes, the script stores the full MAE history, the overall best MAE, beam parameters and the learned misalignment values (where applicable). With a fixed random seed, 10 separate trials are conducted per configuration and the results are saved to CSV file for further analysis.

3.4 Evaluation Metrics

The evaluation code is located in `eval_ares_metrics.py`. For each metric, $i \in \{1, 2, \dots, 10\}$, where i denotes the trial index, $t \in \{0, 1, \dots, T\}$, where t is the step (iteration) index and T is the last step which is 100 in our experiments.

1. **Best MAE:** is the best (minimum) mean absolute error (MAE) reached at any step within each trial, defined as

$$\text{Best MAE}^{(i)} = \min_{t \in \{0, 1, \dots, T\}} \text{MAE}_t^{(i)}. \quad (3.2)$$

2. **Final MAE:** is the MAE at the last step of each trial ($\text{MAE}_T^{(i)}$).
3. **RMSE:** is the root mean square error across all iterations within each trial, capturing how quickly the optimiser improves and not just the final value, defined as

$$\text{RMSE}^{(i)} = \sqrt{\frac{1}{4(T+1)} \sum_{t=0}^T ((\mu_x^{(i,t)})^2 + (\mu_y^{(i,t)})^2 + (\sigma_x^{(i,t)})^2 + (\sigma_y^{(i,t)})^2)}. \quad (3.3)$$

4. **Improvement Best percentage(%):** is the improvement measured from initial to best MAE, computed as $((\text{MAE}_0 - \text{MAE}_{\text{best}})/\text{MAE}_0) \times 100$.
5. **Improvement Final percentage(%):** is the improvement measured from initial to final MAE, computed as $((\text{MAE}_0 - \text{MAE}_{\text{final}})/\text{MAE}_0) \times 100$.
6. **Steps to target:** computes how many evaluations does it take to drop below the threshold of $40 \mu\text{m}$ of MAE for the first time, this shows how quickly an optimiser gets the beam into an acceptable state.
7. **Success rate:** is the percentage of trials that reach the $40 \mu\text{m}$ target within the budget.

The mean, median and standard deviation are computed for each metrics across the 10 trials. Metric definition and evaluation are adapted from [3, 7].

4 Result and Discussion

4.1 Final Results

This section presents the results of comparing four optimisation strategies on the ARES Experimental Area beam tuning problem, including Nelder-Mead, standard BO, BO_prior (mismatched), and BO_prior (matched_prior_newtask). All experiments were run for 100 iterations over 10 independent trials.

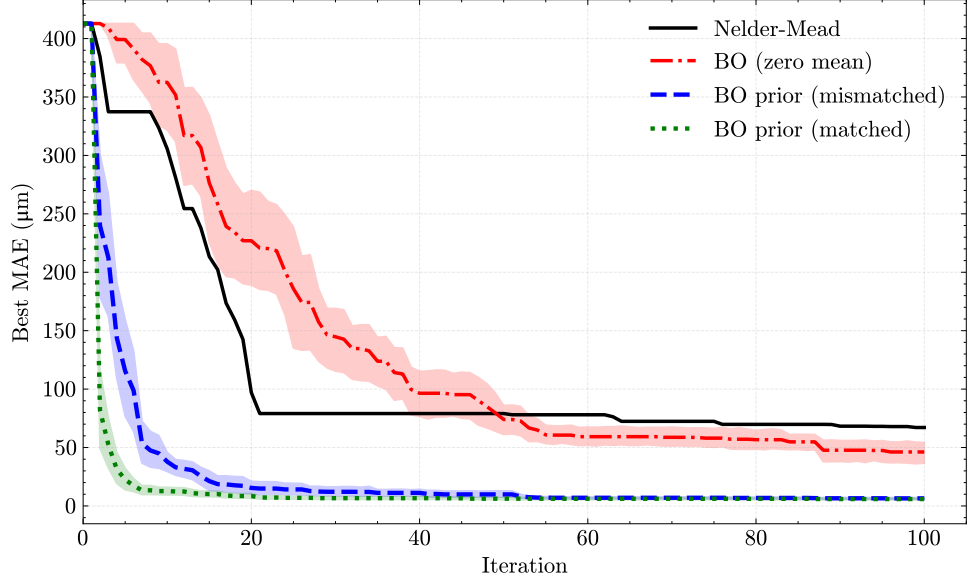


Figure 4.1: Convergence curve with cumulative minimum MAE over iterations.

The convergence curves plot in Figure 4.1 highlights the significantly faster convergence of the two physics-informed BO variants compared to the standard BO and Nelder-Mead baselines. The matched prior (green dotted line) drops sharply within the first 5 iterations, and starts flattening out by iteration 10 already. The mismatched prior (blue dashed line) is only slightly behind the matched case which makes sense because the GP needs some observations before the optimiser can start correcting the misalignment parameters, thus this case takes roughly 20 iterations to reach a comparable level to the matched case. The trajectory of standard BO with zero mean (red dashed-dot line) is more gradual. It does improve steadily, but even by iteration 60, it is still clearly above the physics-informed methods. Additionally, the broad shaded region around, indicates more uncertainty in BO standard case, reflecting the fact that without any guidance from prior, the initial random exploration phase can either go well or poorly, depending on where the points happen to land. Nelder-Mead (solid black line), in the early phase (around iterations 10-20), actually decreases more quickly than standard BO, which is the characteristic of the simplex method to make large directed steps once it finds a downhill direction. However, it plateaus around $65 \mu\text{m}$ and then never goes lower than that. This shows that NM struggles in this five-dimensional space.

Table 5: Performance Metrics Summary (100 iterations, 10 trials, threshold for steps to target and success rate = 40 μm).

Optimiser	Best MAE (μm)		Final MAE (μm)		RMSE (μm)	
	Median	Mean	Median	Mean	Median	Mean
Nelder-Mead	67.13	67.13 $\pm$ 0	67.84	67.84 $\pm$ 0	229.20	229.20 $\pm$ 0
BO (zero mean)	45.88	46.21 $\pm$ 14.70	147.96	153.29 $\pm$ 66.66	844.64	822.19 $\pm$ 118.14
BO + prior (mismatch)	6.71	6.49 $\pm$ 1.11	40.54	41.71 $\pm$ 16.99	219.90	240.16 $\pm$ 65.58
BO + prior (matched)	5.75	5.89 $\pm$ 1.06	42.60	42.09 $\pm$ 16.90	177.57	173.11 $\pm$ 19.34

Optimiser	Improv. Best (%)		Improv. Final (%)		Steps to target		Success rate
	Median	Mean	Median	Mean	Median	Mean	
Nelder-Mead	83.74	83.74 $\pm$ 0	83.57	83.57 $\pm$ 0	—	—	0%
BO (zero mean)	88.89	88.81 $\pm$ 3.56	64.16	62.87 $\pm$ 16.15	82.5	82.5 $\pm$ 5.5	20%
BO + prior (mismatch)	98.37	98.43 $\pm$ 0.27	90.18	89.90 $\pm$ 4.12	10.5	10.8 $\pm$ 4.87	100%
BO + prior (matched)	98.61	98.57 $\pm$ 0.26	89.68	89.81 $\pm$ 4.09	4	3.6 $\pm$ 1.28	100%

Table 5 presents the aggregated performance metrics across all four optimisers evaluated over 10 independent trials of 100 iterations, with a target threshold of $40\text{ }\mu\text{m}$. The results show that both the physics-informed BO variants consistently outperform the baseline approaches across every metric. BO with prior (matched and mismatched) achieve median best MAE values of $5.75\text{ }\mu\text{m}$ and $6.71\text{ }\mu\text{m}$ respectively, which is approximately 10 times lower than Nelder-Mead ($67.13\text{ }\mu\text{m}$) and BO zero mean ($45.88\text{ }\mu\text{m}$). For both matched and mismatched BO prior variants, their improvement percentage (over best MAE) exceeds 98% compared to 83.74% of NM and 88.89% of standard BO. While a gap between best and final MAE is observed for all BO methods (most probably due to continued exploration after finding good solutions), even at the final iteration step, both matched and mismatched physics-informed BO variants maintain lower MAE of $42.60\text{ }\mu\text{m}$ and $40.54\text{ }\mu\text{m}$ respectively than both baseline methods, NM ($67.84\text{ }\mu\text{m}$) and standard BO ($147.96\text{ }\mu\text{m}$). This demonstrates that the physics-informed BO does not only find best solution quickly but also maintain better performance throughout the optimisation, even during exploratory phases. The RMSE values further confirms the advantage of BO with physics-informed prior, with matched prior achieving the lowest RMSE of $177.57\text{ }\mu\text{m}$ reflecting the fact that it starts producing good predictions from the very first steps, followed by the mismatched prior with slightly higher value of $219.90\text{ }\mu\text{m}$ since it spends a few early iterations adjusting its internal model, while standard BO and Nelder-Mead have much higher RMSE values of $844.64\text{ }\mu\text{m}$ and $229.20\text{ }\mu\text{m}$ respectively. This reflects that comparatively to BO with prior variants, both baseline methods have slower convergence and higher errors during the optimisation process.

The efficiency metrics of steps to target and success rate are also impressive for physics-informed BO, comparatively to the NM and standard BO. The median steps to reach the target of $40\text{ }\mu\text{m}$ is just 4 for matched prior and 10.5 for mismatched prior, with 100% success rate for both, while BO zero mean needs 82.5 steps, showing that it meets the target only on rare occasions with success rate of just 20%, whereas Nelder-Mead never reaches the target within the 100 steps budget. This metrics summary provides a comprehensive picture of the significant advantage of using a physics-informed prior in BO for accelerator tuning tasks.

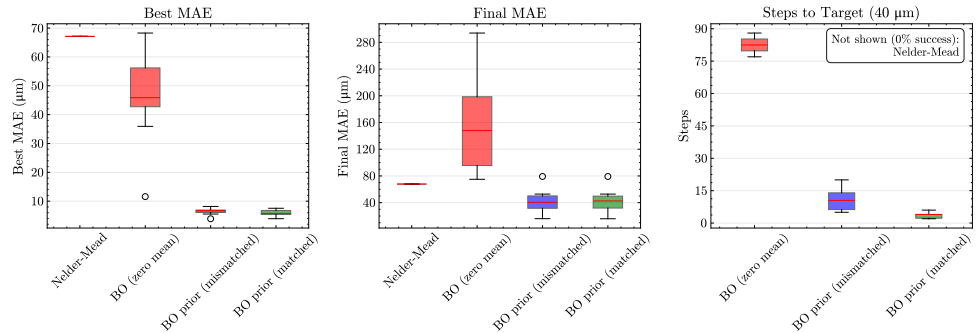


Figure 4.2: Distribution of Best MAE, Final MAE and steps to reach the $40\text{ }\mu\text{m}$ target across 10 trials for each optimiser. the prior-informed BO variants achieve lower error and converge significantly faster than the baseline methods.

The Best MAE panel in Figure 4.2 shows that the prior-informed BO variants cluster

tightly below $10\mu\text{m}$ in best MAE with minimal spread, whereas the standard BO shows high variability ($40\mu\text{m}$ - $56\mu\text{m}$), while NM consistently converges to same local optimum around $67\mu\text{m}$. The final MAE panel highlights the exploration behavior of BO, where standard BO shows the largest spread ($80\mu\text{m}$ - $280\mu\text{m}$), yet the prior-informed BO methods maintain final MAE values around $40\mu\text{m}$, which is still better than both baseline techniques even at the last iteration. The most striking contrast appears in the Steps to Target panel, visually confirming that standard BO requires 75-90 steps to reach the target in very few trials, whereas both BO with prior variants reach the target within 5-15 steps during the trials, confirming that prior knowledge significantly accelerates convergence in addition to improving the solution quality.

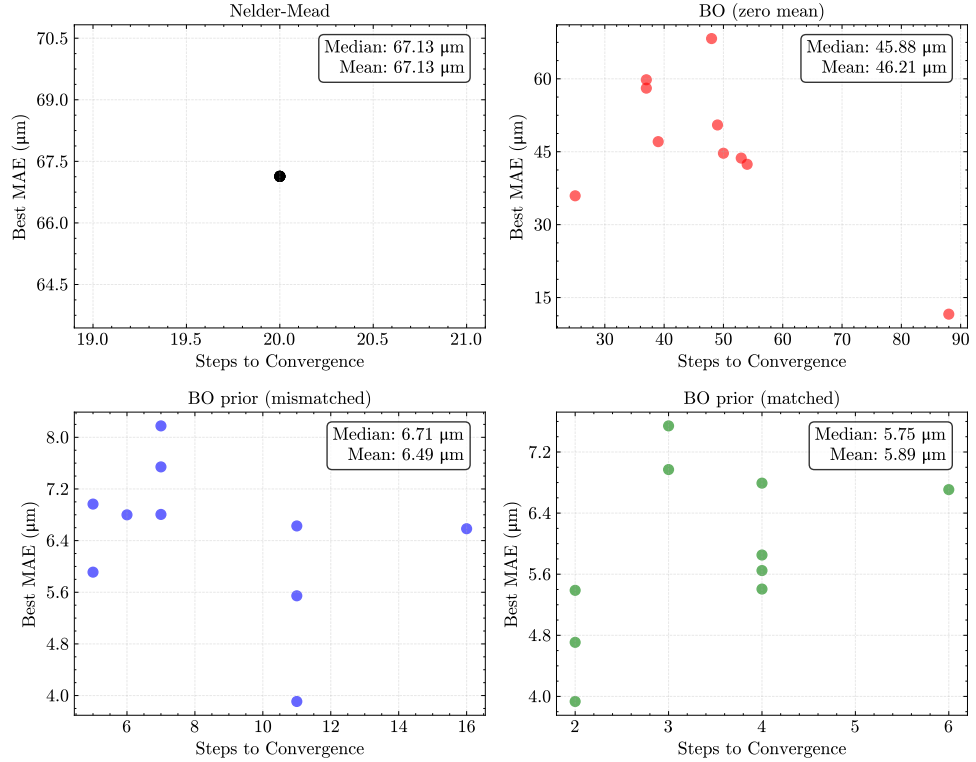


Figure 4.3: Best MAE vs steps to convergence for each trial, shown per optimiser. Each point represents a single trial, with median and mean Best MAE annotated.

The relationship between convergence speed and Best MAE is illustrated by these scatter plots in Figure 4.3. They provide another perspective by plotting each trials's Best MAE against how many steps it took to converge. NM (black dots) clusters at a single point around step 20, but to a poor solution (approx. $67\mu\text{m}$). Standard BO (red dots) shows high variability in both axes, with convergence speed from 25 to 90 steps and Best MAE from 10 to $75\mu\text{m}$, reflecting inconsistency of optimiser without any prior guidance. The prior-informed BO methods occupy the bottom left area of each subplot, which is desirable as it achieves both low (Best) MAE and fast convergence. BO with matched prior (green dots) takes overall 2-6 steps to converge with best MAE from 4 to $8\mu\text{m}$, while BO with mismatched prior (blue dots) takes 5 to 16 steps to a similar quality range as match prior.

Beyond the aggregate performance metrics, it is worth examining whether the mismatched prior can recover the true quadrupole sets solely from the optimisation data. Recalling the ground truth misalignments, the non-zero values were defined in Table 3, while the prior starts with all six values set to zero. By the end of the optimisation process, the optimiser successfully adapts and finds magnet configurations that effectively compensate for the true misalignments, achieving Best MAE values comparable to the matched prior ($6.71\text{ }\mu\text{m}$ vs $5.75\text{ }\mu\text{m}$). This indicates that BO with mismatched prior is able to leverage the optimisation data to adjust its internal model and effectively learn the true misalignment parameters. Thus, demonstrating robustness to prior mismatch. This robustness to prior mismatch is a practical significant finding, as exact knowledge of misalignment is rarely available in real accelerator operations.

4.2 Discussion

All the results and findings portray a consistent picture that Cheetah-based physics prior delivers a significant improvement across every metric, not just in how good the final solution is, but also in how quickly and reliably the optimiser gets there. There are several points worth discussing:

1. GP is given a warm start by the prior mean function, meaning that its predictions already approximate the shape of the true objective. This means that acquisition functions can make intelligent decisions from very first iterations, rather than going through several steps of random exploration. The coverage curve plot (Best MAE) makes this especially clear that physics-informed methods are already below $100\text{ }\mu\text{m}$ by 5 iteration steps, but standard BO takes up to iteration 60 or later. Even in the Final MAE case, the physics-informed BO (matched/mismatched) still outperformed at Final step than standard BO and NM.
2. One might expect mismatched prior BO to perform significantly worse than matched one due to completely incorrect misalignment values given to it. Whereas, the gap is quite small where the median final errors are almost identical, and the mismatched prior only requires approximately 10 extra steps to attain the $40\text{ }\mu\text{m}$ target (10.5 vs 4). This robustness is encouraged, as real world deployments often involve some degree of mismatch too.
3. Though standard BO does eventually reach the threshold of $40\text{ }\mu\text{m}$ with two out of ten trials, but a 20% success rate and a median Steps to Target of 82.5 are not practical for accelerators where the beam is expensive and operators need reliable tuning within few minutes. While, physics-based approaches (matched + mismatched prior) are able to tackle both concerns.
4. The fact that Nelder-Mead converges to the same point in every trial (zero standard deviation on the Best and Final errors), indicates its struggle to handle higher-dimensional problems. In a 5-dimensional space with multiple local minima, this is not unexpected from a convex optimisation algorithm. Thus, neither the Best

MAE ($67.13\ \mu\text{m}$), nor the Final MAE ($67.84\ \mu\text{m}$) by NM is considered acceptable in practice, as it exceeds the $40\ \mu\text{m}$ threshold.

5. Compared to the original FODO demonstration [7], which was a two-dimensional problem with a single learnable parameter, this project work used a five-dimensional landscape with six learnable parameters. Still, the methods with physics-informed prior converged well in approximately 10.5 steps (mismatched prior) or 4 steps (matched prior). This shows that the approach scales reasonably well with the problem dimensionality in this scenario.

5 Conclusion and Outlook

The central question behind this project work is to find out whether a differentiable physics simulator, used as the prior mean in bayesian optimisation (BO), can actually help with a realistic accelerator tuning problem (ARES Experimental Area), or does it only work on a toy example (FODO lattice)? Based on everything presented, including implementation, experiments and the results, it can confidently be said "yes". The problem tackled in this project is the transverse beam tuning task at ARES Experimental Area, the idea of Cheetah-based prior demonstrated on a simple two-dimensional FODO lattice [7] is scaled up to a five-dimensional beam tuning task under this project scope, which includes 5 tunable magnets (three quadrupoles and two correctors), six unknown quadrupole misalignment values that the prior had to learn during optimisation and an objective that asks the optimiser to focus and centre the beam simultaneously. The code is built around a class `AresPriorMean`, that wraps Cheetah and plugs it into the Xopt framework. The approach is tested against three experimental conditions. The matched scenario, where the prior already knows the correct and ideal system state, the mismatched case, where wrong prior is given on purpose, forcing it to learn the correct values during optimisation, and the `matched_prior_newtask` scenario, where all settings (beam and quadrupole misalignment values) are same as mismatched case, but the correct prior is given upfront. For experiments conducted under this project work, the third scenario is considered the actual matched case. Both Cheetah-based BO were compared against two baseline techniques, the standard BO with zero prior and Nelder-Mead Simplex.

The results confirm that using physics-informed prior in BO, significantly improves the performance. Both the BO prior variants (matched and mismatched) achieved a median Best MAE of $5.75\,\mu\text{m}$ and $6.71\,\mu\text{m}$ respectively, which is significantly better than standard BO ($45.88\,\mu\text{m}$) and Nelder-Mead($67.13\,\mu\text{m}$). BO with both matched and mismatched prior also achieved better Final MAE compared to baseline methods, with a median of $42.60\,\mu\text{m}$ and $40.54\,\mu\text{m}$ respectively, while standard BO and Nelder-Mead had a median Final MAE of $147.96\,\mu\text{m}$ and $67.84\,\mu\text{m}$ respectively. The matched prior hit the $40\,\mu\text{m}$ target in a median of 4 steps, the mismatched prior in 10.5, whereas standard BO required 82.5 steps on rare occasions (20% success rate) and Nelder-Mead never reached the target. What makes the approach especially promising for real world use is its robustness. Even when starting with incorrect misalignment values and modified beam distribution, the prior still outperformed the standard BO and reached on a comparable level as the matched prior scenario of BO very soon within 15-20 iterations. The trainable misalignment parameters successfully converged very close to the ground truth purely from optimisation data, which means that the method does not just tune the beam but it also learns something about the machine's physical state along the way.

Ofcourse, the ARES Experimental Area is still a relatively small problem compared to large-scale facilities like European XFEL, but this is a significant step forward from the two-dimensional FODO lattice that is previously tested to evaluate cheetah's potential. The fact that the method scales gracefully to this level sends quite good signs for scaling further and handling more complex beamlines in future.

6 Future Work

This project work has the potential to be extended in several directions ranging from immediate next steps to longer term research questions. The most obvious next step now would be to test the whole implementation on the real ARES accelerator rather than in simulation. A real machine could bring things that a simulator cannot completely capture, like measurement noise, small differences between cheetah model and actual machine etc. Another interesting step for the future work could be to convert the code into a Python package. The system is currently tailored to ARES Experimental Area, with hardwired magnets names, lattice file, and misalignment values. But the idea of picking a cheetah lattice, tuning, learning and running BO itself is not specific to any one beamline, so packaging it up with a cleaner interface would make it much easier for others to try on their own accelerators.

Currently, each time the optimizer runs, it starts fresh, but in real accelerators the machine does not change drastically between the tuning sessions, The learned misalignment values and GP hyperparameters could be used as warm start for the next, potentially speeding up the convergence even further. So it would be worth having a look into it more. Some other research direction that could push this project further for future work could be tunable parameters increment. The ARES problem only has 5 tunable parameters that were the 5 magnets, but a real beamline can have dozens of them. At such scale, GP gets expensive and it gets harder for acquisition function to search, so combining Cheetah prior with dimensionality reduction techniques would be worth exploring.

The current project uses objective function that combines beam position and size into a single MAE metric. In some operational scenarios, it might be more natural to treat these as separate objectives (centering the beam and focusing the beam) and use multi-objective BO instead of combining everything to a single MAE, so this could be another direction to experiment. Furthermore, on the modeling side, only the quadrupole misalignments were learned in this project, but there could also be other sources mismatched/unknown in the real machine, like incoming beam distribution may not be perfectly known, thus experiments can be conducted to learn other unknown parameters as well. Finally, the tests could also be conducted on more advanced beamlines with more complex objectives like the ones used in European XFEL.

References

- [1] Deutsches Elektronen-Synchrotron DESY. “Desy - homepage”. [Online]. Available: <https://desy.de/>
- [2] J. A. Nelder and R. Mead, “A simplex method for function minimization”, *The Computer Journal*, 1965.
- [3] J. Kaiser et al., “Reinforcement learning-trained optimisers and bayesian optimization for online particle accelerator tuning”, *Scientific reports*, 2024.
- [4] R. Roussel et al., “Bayesian optimization algorithms for accelerator physics”, *Physical review accelerators and beams*, 2024.
- [5] J. Alas et al., “Bayesian optimization of a laser-plasma accelerator”, *Physical review letters*, 2021.
- [6] J. Duris et al., “Bayesian optimization of a free-electron laser”, *Physical review letters*, 2020.
- [7] J. Kaiser et al., “Bridging the gap between machine learning and particle accelerator physics with high-speed, differential simulations”, *Physical review accelerators and beams*, 2024.
- [8] T. Boltz et al., “Leveraging prior mean modules for faster bayesian optimization of particle accelerators”, *Scientific Reports*, 2025.
- [9] L. Deniau, *Methodical accelerator design/mad-x: 5.09.01*, 2023. DOI: 10.5281/zenodo.10255261
- [10] M. Borland, “Elegant: A flexible sdds-compliant code for accelerator simulation”, Advanced Photon Source, Tech. Rep., 2000.